



Día, Fecha:	DIA, DD/MM/YYYY
Hora de inicio:	00:00

Sistemas Operativos 2 [A]

NOMBRE_ESTUDIANTE

UNIDAD 4: SISTEMAS ESPECIALIZADOS Y DISTRIBUIDOS

Escuela de Ingeniería de Ciencias Y Sistemas
Facultad de Ingeniería
Universidad de San Carlos de Guatemala



Anuncios Importantes

Tema 1

Tema 2

Tema 3

Tema 4



AGENDA



1. Características del sistema operativo móvil (Android, iOS)
2. Características del sistema operativo integrado (FreeRTOS)
3. Restricciones de recursos en sistemas integrados
4. Compilación cruzada y desarrollo para plataformas integradas
5. Llamadas a procedimientos remotos (RPC)

COMPETENCIA(S) QUE DESARROLLAREMOS



- Entender el marco de referencia o estructura lógica general de un sistema operativo, que le permita la utilización, análisis y diseño de sistemas operativos.
- Evaluar sistemas operativos para soluciones específicas.

LABORATORIO SISTEMAS OPERATIVOS 2

SISTEMAS OPERATIVOS MÓVILES E INTEGRADOS

SO MÓVILES - INTEGRADOS



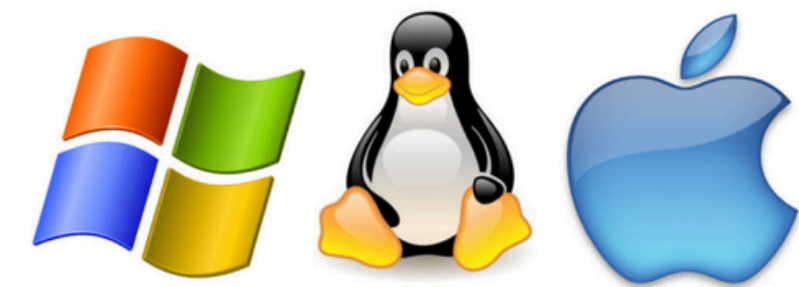
Sistemas Operativos Móviles

Son plataformas de software diseñadas para gestionar y operar el hardware de dispositivos móviles, como smartphones y tablets. Proporcionan el entorno necesario para la ejecución de aplicaciones y la interacción del usuario

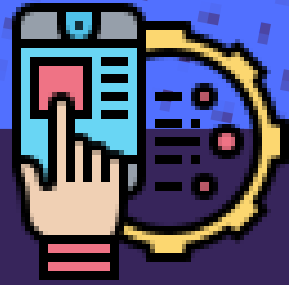


Sistemas Operativos Integrados

Son sistemas diseñados para controlar dispositivos específicos y realizar tareas dedicadas en tiempo real. Se centran en la eficiencia y la optimización de recursos, siendo comunes en electrodomésticos inteligentes, sistemas de control industrial y dispositivos IoT.



CARACTERÍSTICAS



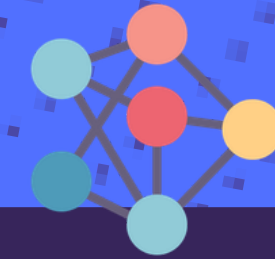
Interfaz de Usuario:

- Los sistemas operativos móviles suelen ofrecer interfaces gráficas ricas y táctiles, facilitando la interacción del usuario a través de pantallas sensibles al tacto.
- En contraste, los sistemas operativos integrados pueden operar con interfaces más simples o incluso sin interfaz, ya que están diseñados para ejecutar funciones específicas sin necesidad de interacción directa del usuario.



GESTIÓN DE RECURSOS

- Los sistemas móviles deben gestionar múltiples aplicaciones y servicios, lo que requiere una planificación y asignación de recursos eficiente para asegurar un rendimiento óptimo.
- Los sistemas integrados optimizan sus recursos para tareas específicas, priorizando la rapidez y eficiencia en la ejecución de funciones dedicadas.



Conectividad

- Los sistemas móviles están diseñados para conectarse a diversas redes (Wi-Fi, Bluetooth, 4G/5G), permitiendo una comunicación constante y el acceso a servicios en la nube.
- En comparación, los sistemas integrados suelen estar conectados a través de interfaces específicas (como UART, SPI, I2C), adecuadas para la comunicación con otros dispositivos dentro de un sistema más grande.

ÁREAS DE APLICACIÓN

01



DISPOSITIVOS DE CONSUMO

Experiencia del Usuario: Interfaz gráfica intuitiva y personalizable.

Multitarea: Capacidad para ejecutar múltiples aplicaciones simultáneamente, permitiendo alternar entre tareas sin problemas.

Acceso a Servicios en Línea: Integración con aplicaciones de redes sociales, mensajería, y servicios de streaming.

02



AUTOMOCIÓN

En el sector automotriz para:

Sistemas de Navegación: GPS y aplicaciones de mapas

Conectividad: Integración con smartphones.

Seguridad: Sistemas de control de estabilidad, asistencia al conductor, etc

03



DISPOSITIVOS MÉDICOS

Monitores de Salud: Dispositivos que supervisan constantemente parámetros vitales, como la frecuencia cardíaca y la presión arterial.

Interfaz de Usuario: Los dispositivos a menudo incluyen pantallas táctiles que permiten a los médicos y pacientes interactuar con el sistema de manera intuitiva

04

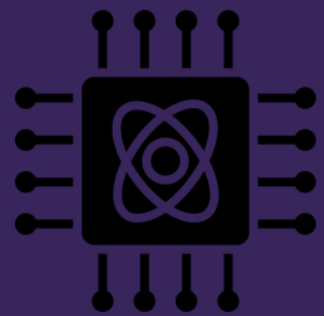


IOT

Hogares Inteligentes: Dispositivos como termostatos inteligentes, cámaras de seguridad, y asistentes virtuales que gestionan la eficiencia energética, seguridad, y comodidad del hogar.

Ciudades Conectadas: Sistemas de gestión del tráfico, monitorización ambiental, y servicios públicos que utilizan sensores y dispositivos conectados para mejorar la infraestructura urbana.

SISTEMA OPERATIVO MÓVIL (ANDROID)



Kernel

Se basa en el núcleo de **Linux**, que es responsable de gestionar el hardware, los procesos, la memoria y la seguridad del sistema. Esta base proporciona una sólida estabilidad y seguridad, que es crucial para un sistema operativo móvil.



Bibliotecas

Android utiliza bibliotecas escritas en C++ y Java, que ofrecen una serie de API (Interfaces de Programación de Aplicaciones) que los desarrolladores pueden utilizar para crear aplicaciones que aprovechen las características del hardware del dispositivo.

Android fue lanzado en 2008 por Google y rápidamente se convirtió en el **sistema operativo** móvil más utilizado a nivel mundial, impulsado por su naturaleza de código abierto y su amplia variedad de dispositivos que lo utilizan.

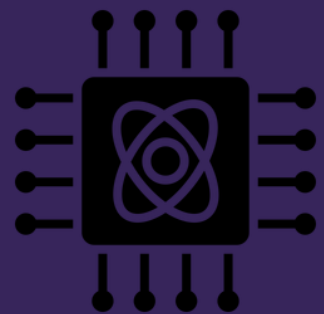


Herramientas

Android Studio es el entorno de desarrollo integrado (IDE) oficial, y Gradle es el sistema de automatización de compilación utilizado para gestionar las dependencias y el proceso de construcción de aplicaciones.

Kotlin y Java son los principales **lenguajes** de programación utilizados en el desarrollo de aplicaciones para Android. Kotlin, en particular, ha ganado popularidad por su concisión y seguridad.

SISTEMA OPERATIVO MÓVIL (IOS)



KERNEL

El núcleo de iOS es XNU, que combina elementos del sistema operativo **Mach** y del sistema BSD. Esto le proporciona un alto rendimiento y una sólida seguridad.



Frameworks

Cocoa Touch es el framework principal que proporciona los componentes necesarios para crear interfaces de usuario y gestionar eventos. Incluye herramientas para crear aplicaciones responsivas y visualmente atractivas.

IOS fue introducido en 2007 con el lanzamiento del iPhone, desarrollado por **Apple**. Desde entonces, ha evolucionado para convertirse en un sistema operativo potente y muy optimizado para dispositivos móviles, como iPhone y iPad.



Herramientas y lenguajes

- **Xcode** es el IDE utilizado para desarrollar aplicaciones iOS, que incluye un conjunto completo de herramientas para crear, depurar y probar aplicaciones. SwiftUI es un framework que facilita la creación de interfaces de usuario modernas y adaptativas.
- **Swift** es el lenguaje principal para el desarrollo de aplicaciones en iOS, aunque Objective-C sigue siendo compatible. Swift es conocido por su sintaxis clara y su enfoque en la seguridad.

SISTEMAS OPERATIVOS INTEGRADOS (FREERTOS)

Descripción general

FreeRTOS es un sistema operativo de tiempo real (RTOS) de código abierto, diseñado específicamente para microcontroladores y microprocesadores. Su propósito es proporcionar un entorno eficiente y flexible para aplicaciones que requieren un comportamiento en tiempo real, donde los plazos y la respuesta rápida son críticos.



01

MULTITAREA

Admite la ejecución de múltiples tareas con diferentes niveles de prioridad. Esto permite que las tareas críticas sean procesadas en tiempo real, garantizando que los sistemas reaccionen rápidamente a eventos importantes sin retrasos.

02

GESTION DE RECURSOS

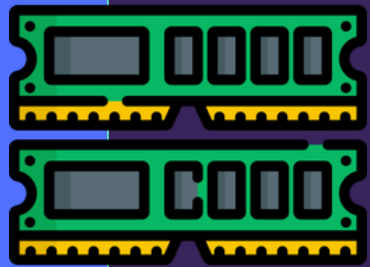
Gestiona eficientemente los recursos del sistema, incluyendo memoria y mecanismos de sincronización como **semáforos** y mutexes, que coordinan el acceso a recursos compartidos entre tareas, previniendo condiciones de carrera y asegurando la integridad de los datos.

03

ESCALABILIDAD

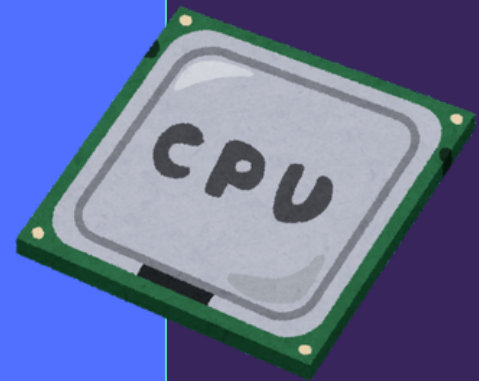
FreeRTOS es altamente escalable, lo que permite su uso en aplicaciones que van desde sistemas simples con pocas tareas hasta sistemas complejos que gestionan múltiples tareas simultáneas,

RESTRICCIONES DE RECURSOS EN SISTEMAS INTEGRADOS



Memoria

Suelen contar con capacidad limitada de RAM, que a menudo se mide en kilobytes o pocos megabytes. Esto obliga a los desarrolladores a optimizar el uso de memoria, evitando la carga innecesaria de datos y limitando el tamaño de las aplicaciones.



CPU

Suelen utilizar microcontroladores que tienen capacidades de procesamiento reducidas, lo que se traduce en frecuencias de reloj más bajas y arquitecturas simplificadas.



Energía

Muchos sistemas integrados, especialmente en el contexto de dispositivos portátiles o de Internet de las Cosas (IoT), están diseñados para funcionar con bajo consumo de energía. La duración de la batería es un factor crítico, lo que requiere que el diseño del sistema tenga en cuenta la eficiencia energética en cada etapa del desarrollo.

ESTRATEGIAS DE DESARROLLO



OPTIMIZACIÓN DEL CÓDIGO

Para enfrentar las limitaciones de memoria y CPU, es crucial utilizar algoritmos y estructuras de datos que sean eficientes. Esto incluye evitar bucles innecesarios y seleccionar algoritmos que ofrezcan la mejor relación de rendimiento en función de los recursos disponibles.



USO DE RECURSOS

Se recomienda minimizar el uso de bibliotecas de **gran tamaño** que puedan ocupar una cantidad significativa de memoria. En su lugar, se deben optar por funciones específicas y optimizadas que se alineen con las necesidades del proyecto.



GESTIÓN DE ENERGÍA

Implementar **modos de sueño** y técnicas de gestión de energía puede ser vital para prolongar la vida útil de la batería. Esto incluye la posibilidad de poner el dispositivo en estado de bajo consumo cuando no está en uso o ajustar dinámicamente la frecuencia del procesado

COMPILACIÓN CRUZADA Y DESARROLLO PARA PLATAFORMAS INTEGRADAS

La compilación cruzada es el proceso de construir ejecutables en un sistema host (la máquina en la que se desarrolla) para que estos ejecutables funcionen en un sistema objetivo (target) que generalmente tiene una arquitectura diferente. Este proceso es común en el desarrollo de sistemas embebidos, ya que los dispositivos objetivos a menudo cuentan con microcontroladores de arquitecturas específicas.



COMPILADORES

- **GCC (GNU Compiler Collection)** para ARM: Amplia compatibilidad y es de código abierto.
- **IAR Embedded Workbench:** Ofrece herramientas de desarrollo optimizadas para sistemas embebidos.
- **Keil uVision:** Muy usado en entornos de microcontroladores, especialmente en el desarrollo de aplicaciones ARM.

ENTORNOS DE DESARROLLO



- **PlatformIO:** Un IDE multiplataforma que facilita la configuración de proyectos y soporta múltiples frameworks y arquitecturas.
- **Eclipse con CDT:** Eclipse, junto con el complemento CDT (C/C++ Development Tooling), permite el desarrollo cruzado con soporte para varios compiladores y herramientas.
- **Visual Studio Code:** Un editor ampliamente usado que, con extensiones, facilita el desarrollo para plataformas embebidas.

PROCESO DE COMPILACIÓN CRUZADA

CONFIGURACIÓN DEL ENTORNO

Se instalan las herramientas de compilación necesarias (como el compilador cruzado) y se configuran en el sistema host. Este paso implica especificar los caminos y las banderas necesarias para el compilador, dependiendo de la arquitectura del dispositivo objetivo.

DESARROLLO DE CÓDIGO

El código se escribe en un lenguaje compatible (habitualmente C o C++), teniendo en cuenta las limitaciones del sistema embebido y optimizando para el tamaño y eficiencia del binario.

COMPILACIÓN

Se ejecuta el compilador cruzado, configurado con las banderas y opciones adecuadas, para generar un archivo binario ejecutable compatible con el dispositivo objetivo.

CARGA Y PRUEBA

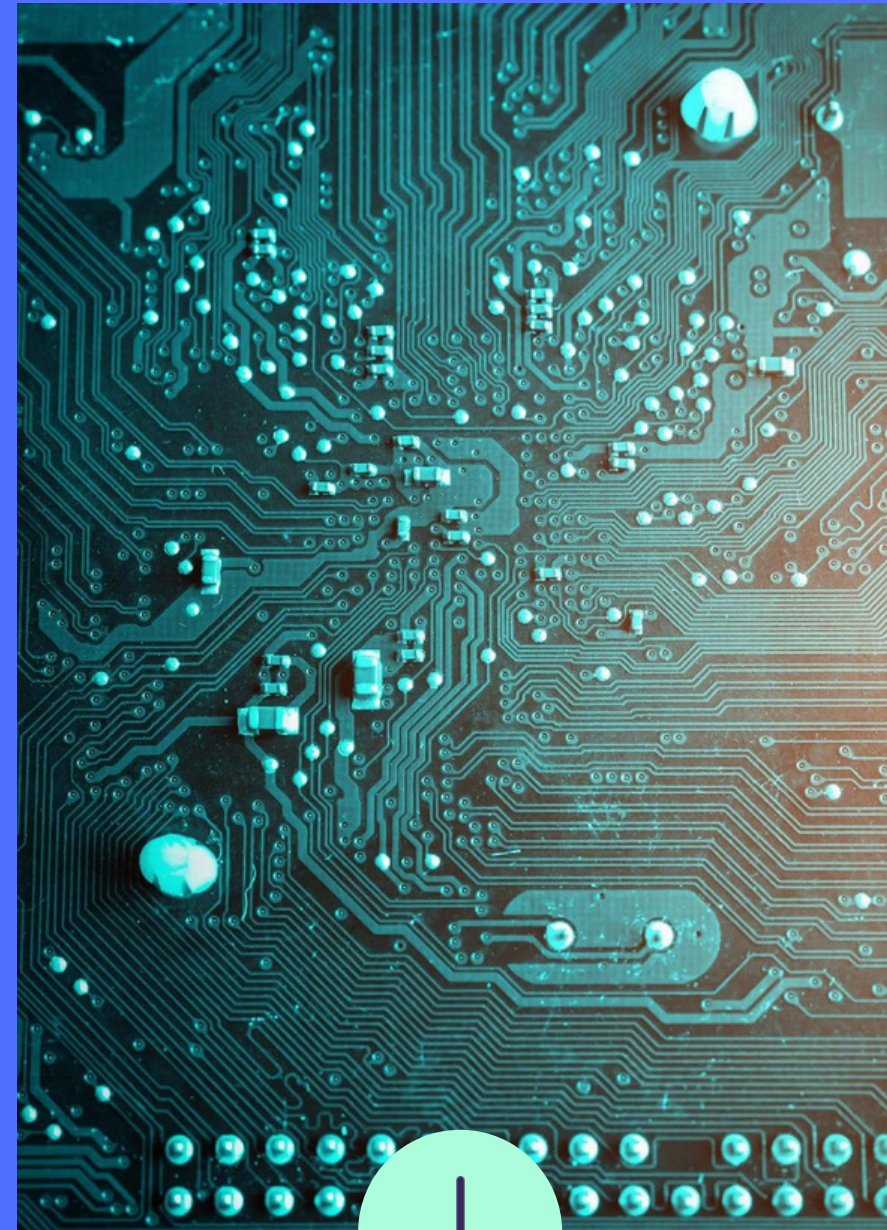
Una vez generado el binario, se transfiere al dispositivo mediante USB, JTAG, o a través de un puerto de serie. Luego, se realizan pruebas para asegurar que el software funcione correctamente en el hardware de destino, y se depuran posibles errores.

LLAMADAS A PROCEDIMIENTOS REMOTOS (RPC)

Las Llamadas a Procedimientos Remotos (RPC, por sus siglas en inglés) son un mecanismo que permite a un programa ejecutar una función o procedimiento en una computadora remota como si fuera una llamada local. Este enfoque facilita la comunicación y la colaboración entre diferentes sistemas dentro de un entorno distribuido.

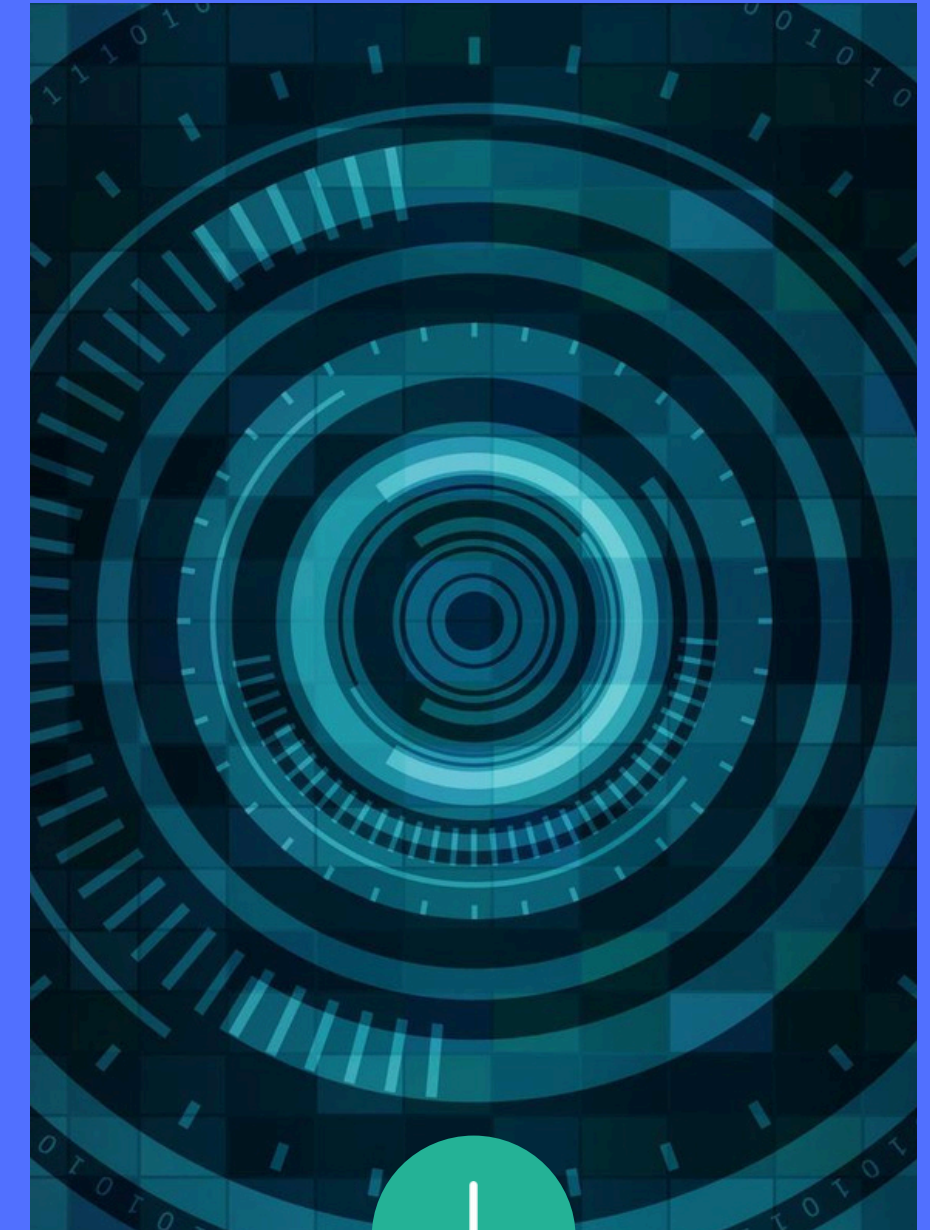
OBJETIVO DE LAS RPC

- Simplificar la interacción entre sistemas distribuidos.
- Ocultar la complejidad de la comunicación de red al programador.



¿CÓMO FUNCIONA?

- El cliente invoca una función local que, en realidad, envía una solicitud al servidor.
- El servidor ejecuta la función solicitada y devuelve el resultado al cliente.



IMPORTANCIA EN LOS SISTEMAS DISTRIBUIDOS

- Permiten la construcción de aplicaciones distribuidas modulares y escalables.
- Facilitan la reutilización de servicios y recursos en diferentes partes de una red.

FUNCIONAMIENTO BÁSICO DE RPC

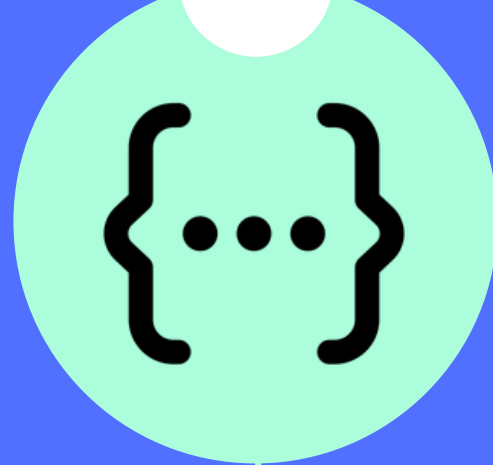
01



CLIENTE HACE UNA SOLICITUD

- El cliente invoca una función local que actúa como un proxy.
- **Stub del Cliente:** Una interfaz que prepara la solicitud para enviarla al servidor.

02



MARSHALLING

Serialización de los parámetros de la función para su transmisión a través de la red.

03



TRANSMISIÓN DE LA SOLICITUD

Los datos serializados se envían al servidor a través de la red.

04

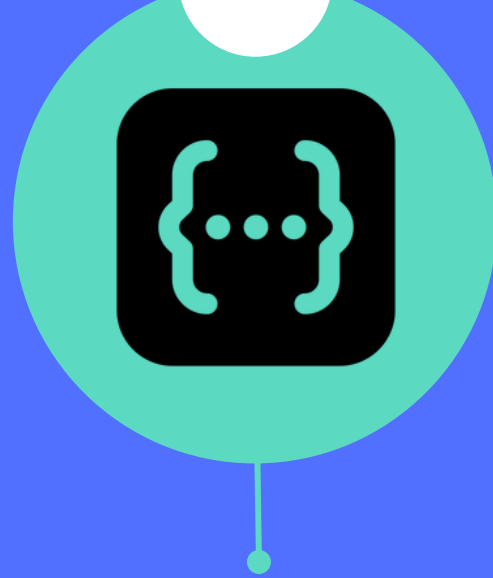


SERVIDOR PROCESA LA SOLICITUD

Stub del Servidor: Recibe la solicitud, deserializa los parámetros y ejecuta la función solicitada.

FUNCIONAMIENTO BÁSICO DE RPC

05



UNMARSHALLING

Deserialización del resultado para enviarlo de vuelta al cliente.

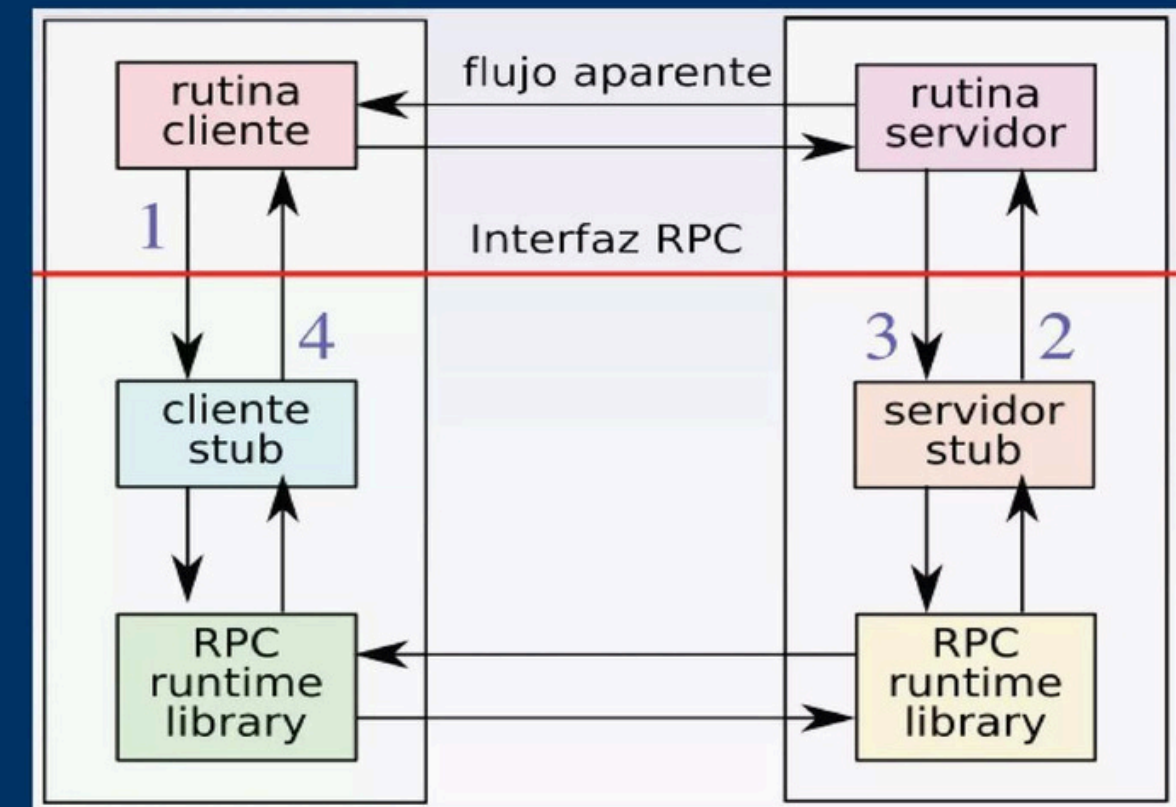
06



RESPUESTA AL CLIENTE

El cliente recibe el resultado y lo procesa como si fuera una llamada local.

Modelo RPC



- **Marshalling:** Proceso de convertir los parámetros de la función en un formato que pueda ser transmitido a través de la red.
- **Unmarshalling:** Proceso de convertir los datos recibidos de vuelta a su formato original para su uso en el programa.

PROTOCOLOS DE **RPC**



gRPC

Un framework de RPC de alto rendimiento desarrollado por Google, basado en HTTP/2.

- Soporte para múltiples lenguajes, streaming bidireccional, eficiente en el uso de recursos.
- **Casos de Uso:** Microservicios, comunicación entre servicios en la nube.



SOAP (Simple Object Access Protocol):

Un protocolo basado en XML para intercambiar información estructurada en la implementación de servicios web.

- Extensible, independiente del protocolo de transporte, alto nivel de seguridad.
- **Casos de Uso:** Servicios empresariales, aplicaciones que requieren transacciones seguras y fiables.



XML-RPC

Un protocolo que utiliza XML para codificar sus llamadas y HTTP como protocolo de transporte.

- Simplicidad, fácil de implementar.
- **Casos de Uso:** Integración de sistemas, aplicaciones web simples.



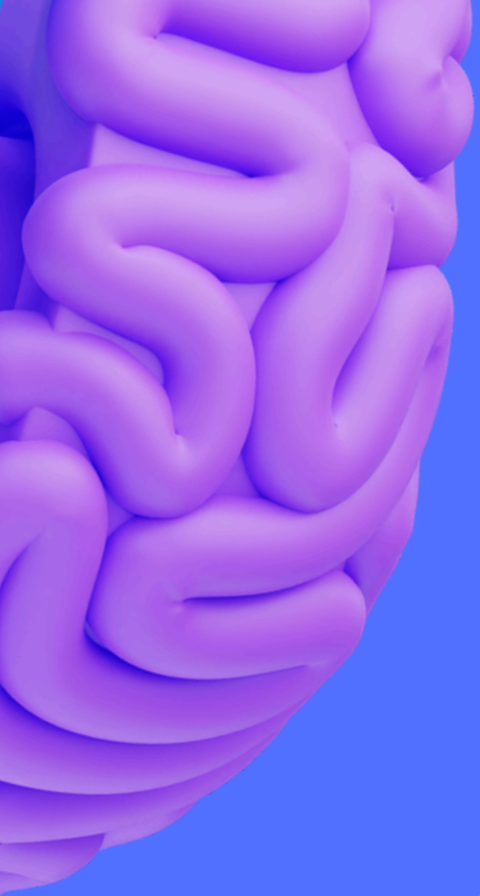
JSON-RPC

Similar a XML-RPC pero utiliza JSON para la codificación.

- Ligero, fácil de leer y escribir, compatible con aplicaciones web modernas.
- **Casos de Uso:** Aplicaciones web y móviles, APIs RESTful.

¿CUÁL USAR?

- **gRPC:** Comunicación entre microservicios en una arquitectura de Kubernetes.
- **SOAP:** Servicios web financieros que requieren alta seguridad y transacciones confiables.
- **JSON-RPC:** APIs para aplicaciones móviles que requieren respuestas rápidas y ligeras.



Ejemplo Guiado:

gRPC

CONCEPTOS CLAVE APRENDIDOS

- **Sistemas Operativos Móviles e Integrados:** Los móviles gestionan dispositivos con interfaces gráficas y redes, mientras que los integrados controlan dispositivos específicos con recursos limitados y tareas en tiempo real (ej. Android vs FreeRTOS).
FreeRTOS y Restricciones: FreeRTOS es un RTOS ligero para microcontroladores que optimiza multitarea y recursos en entornos con memoria, CPU y energía muy limitadas.
- **RPC:** RPC permite llamar funciones remotas como locales, usando protocolos como gRPC, SOAP o JSON-RPC para facilitar la comunicación en sistemas distribuidos.

VALOR DE LA SEMANA

- **Innovacion:** Al entender las limitaciones de la tecnología, es inevitable buscar formas alternativas de obtener resultados, las computadoras han evolucionado muchas veces a lo largo de la historia, lo mismo ha sucedido con los sistemas operativos, de manera que es necesario continuar esa evolución e innovación.

REFERENCIAS

FreeRTOS

<https://freertos.org/>

Optimizing Embedded Systems

<https://emb-academy.com/articles/optimizing-performance-in-embedded-systems/>

gRPC

<https://grpc.io/>

¡GRACIAS POR
LA ATENCIÓN!

¿Dudas?